

Enterprise System Integration

Project Guidelines 2026

Total Points: 30

Deadlines:

Checkpoint #1 (**12 points**): **05 May** 2026

Checkpoint #2 (**8 points**): **12 May** 2026

Checkpoint #3 & **Final Submission** (**10 points**): **19 May** 2026

Note: At each checkpoint, the last commit allowed in the repository is at 14:00 (Estonian time) on the corresponding day. The discussions start on the same day at 14:15, during the lab sessions. Every team will have 15 minutes for discussions and book their preferred spot in a poll shared on Moodle.

1. Objective

The goal of this project is to implement a microservices-based system based on your previous assignments.

You are not designing a new system. You are implementing:

- Your proposed scenario from **Assignment 1**,
- the domain model from **Assignment 2**,
- the APIs, architecture, data models, and workflows from **Assignment 3**.

The focus is on:

- correct service implementation,
- proper architecture (Spring Boot layering),
- service integration (REST + async),
- frontend interaction,
- basic security.

2. Individual Responsibilities

This project is graded individually, and each student must implement either 2 microservices, or 1 microservice + 1 integration/resilience component.

Examples of integration components:

- API Gateway
- Messaging/event infrastructure
- Service discovery / load balancing
- Resilience mechanisms
- Any other you considered relevant and declared in **Assignment 3**.

Important: Your responsibilities are fixed after your **Assignment 3**. If you want to change something, please contact Orlenys via Slack.

3. Technical Requirements

3.1 System Requirements - the final system must include:

- multiple interacting services
- a frontend application
- basic security (authentication and authorization)
- all services running together (e.g., Docker)

3.2. Microservices Requirements - each implemented service must be autonomous, deployable independently, and:

- have a layered structure:
 - Controller (API)
 - DTO layer
 - Service (business logic)
 - Repository (persistence)
 - Domain model
- manage its own database
- expose REST endpoints (~5–8 endpoints expected)
- have at least one asynchronous workflow (events/messaging)

Controller Layer (API) - responsible for:

- exposing REST endpoints
- receiving HTTP requests
- validating request input
- returning HTTP responses

The controller should not contain business logic or database access.

DTO Layer - responsible for:

- request DTOs
- response DTOs
- separating the API contract from the internal domain model

DTOs must be used to:

- receive data from clients
- return data to clients
- avoid exposing entities directly through the API

The API model and the persistence/domain model should not be tightly coupled.

Service Layer (Business Logic) - responsible for:

- implementing use cases
- enforcing business rules
- coordinating repository access
- coordinating interaction with other services/components

The service layer should contain the main application logic.

Repository Layer (Persistence) - responsible for:

- database access

- storing and retrieving entities
- encapsulating persistence operations

Repositories should not contain business logic.

Domain Model - responsible for:

- representing the main business concepts of the service
- entities
- value objects
- core domain data

The domain model should reflect the service data model defined in Assignment 3.

Simple Flow

A well-structured request flow should look like this:

Client → Controller → Service → Repository → Database

And for responses:

Database → Repository → Service → DTO mapping → Controller → Client

4. Detailed Timeline & Checkpoints

4.1. Checkpoint 1 — Backend: Service 1 Complete (12 points)

Goal: Deliver one **fully implemented, testable, and documented service.**

IMPORTANT: 1 service per student (4 or 5 in total depending on team size). Each student will be graded individually, based on their implemented microservice.

Required Deliverables

- Running Service:** Service starts and endpoints are accessible.
- API Implementation:** All endpoints implemented according to **Assignment 3**.
- OpenAPI / Swagger:** Swagger UI available, with endpoints documented and testable.
- Persistence:** Database connected, and some data stored and retrieved.
- Testing (at least 1 endpoint interacting with another service or component is required)**
 - At least one @WebMvcTest test class (or any other testing framework you prefer).
 - At least: one successful (happy path) test, and one error case test.
 - The tested endpoint must involve a dependency on another component/service.
 - Dependencies may be mocked.
 - The test must show that the service behaves correctly based on the mocked response.
- API Demonstration:** Show working endpoints using Postman or Swagger.

Grading:

- Full service implemented, excluding asynchronous communication (A, B and D): 7 points
- Swagger/OpenAPI (C): 2 points
- Testing (E): 2 points
- API demo (F) 1 point

Note: A service is considered fully implemented if all endpoints defined in **Assignment 3** are available and functional.

4.2. Checkpoint 2 — Service 2 / Integration + Frontend (8 points)

Goal: Complete backend responsibilities and start system integration, without considering security. This does not mean implementing security mechanisms is forbidden at this checkpoint. You can include security mechanisms if you want, but it will be assessed in the last checkpoint.

Requirement (per student): You must show your second service implemented, or your integration/resilience component implemented and usable.

IMPORTANT: For the second service or resilience mechanism, you do **NOT** need to include any tests and documentation. In other words, just implement the points A, B, and D applied to service implementation in Checkpoint 1. For the API demonstration (point F in checkpoint 1), you can use Postman or your front-end for the already implemented endpoints.

Required Deliverables

- A. Second Responsibility:** Second service OR integration component runs.
- B. Basic Integration:** At least one working interaction between two implemented services must be demonstrated (real call, not mocked).
- C. Frontend:** The frontend exists. It calls at least one backend endpoint (per student) and displays real data.

Grading:

- Second service/component (A): 4 points
- Basic Integration: 2 pts
- Initial Frontend: 2 points

4.3. Checkpoint 3 — Final System (10 points)

Goal: Deliver a **complete, integrated system**

Required Deliverables

- A. All Responsibilities Completed:** Each student either completed 2 services, or 1 service + 1 integration component.
- B. Full System Running:** All services run together, e.g., Docker Compose.
- C. Frontend:** Fully functional, i.e., all the endpoints of the implemented microservices can be triggered from the frontend. It handles errors in responses from the backend.
- D. Security:** Authentication and authorization implemented and applied to endpoints.
- E. Asynchronous Communication:** At least one event/message-based interaction between services.

Grading:

- Responsibilities/integrations completed, including async communication (A and E): 3 points
- System runs in Docker (B): 2 pts
- Frontend completed (C): 2 pts
- Security (D): 3 pts